

Dispositivos Conectados

ESP32, *toolchain* ESP-IDF e Input/Output Básico

2026/2027

Author: Paulo C. Bartolomeu

1 Objetivos

- **Parte 1**
 - Familiarização com a placa ESP32
 - Instalação e familiarização com a *framework* ESP-IDF
 - Criação e execução de um projeto “Hello World”
- **Parte 2**
 - Configuração e utilização de portos de saída digital;
 - Configuração e utilização de portos de entrada digital (*polling* e interrupção);
 - Configuração e utilização de *Queues* para comunicação entre tarefas;
 - Configuração e geração de sinais PWM;
 - Configuração e leitura de portos de *input* analógico.

2 Introdução

Num contexto de Internet das Coisas (IoT), a capacidade de controlar e ler sinais elétricos através dos portos de entrada e saída digital e analógica é fundamental para criar dispositivos inteligentes e interativos. Saber programar estes portos permite que um microcontrolador se comunique com sensores, atuadores e outros periféricos, transformando sinais do mundo físico em dados digitais que podem ser processados e transmitidos, ou convertendo comandos digitais em ações concretas, como acender uma luz, movimentar um motor ou regular a intensidade de um LED. Esta habilidade não só proporciona um controlo direto sobre o ambiente, mas também abre caminho à automação, à monitorização em tempo real e à integração de múltiplos dispositivos numa rede IoT. Compreender o funcionamento e a programação dos portos de I/O é, portanto, essencial para desenvolver soluções robustas, eficientes e responsivas, permitindo aos criadores transformar ideias em aplicações funcionais que interagem de forma inteligente com o mundo ao seu redor.

Nesta aula iremos introduzir a *framework* de *hardware* e *software* a usar nesta UC, assim como explorar a sua utilização para efetuar operações de controlo sobre um LED, ler o estado de um botão de pressão e monitorizar a tensão de um divisor resistivo implementado com um potenciômetro. Tratam-se de operações básicas de I/O, mas que são fundamentais para a construção de um sistema integrados IoT.

3 Parte 1: Familiarização com a framework de *hardware* e *software*

3.1 Kits de desenvolvimento ESP32

As aulas práticas da UC de Dispositivos Conetados suportam-se na utilização de um módulo ESP32-C6-DevKitC-1 da Espressif Systems para o desenvolvimento de soluções IoT (Internet das Coisas). Este módulo de elevado desempenho possui recursos avançados de conectividade (Wi-Fi, Bluetooth LE, Zigbee e Thread), proporcionando uma plataforma robusta para criar uma ampla gama de aplicações. O módulo ESP32-C6-DevKitC-1 é fornecido num kit IoT constituído pelos componentes apresentados na Tabela 1:

Tabela 1: Kit Base.

Componente	Descrição
ESP32-C6-DevKitC-1	Placa de desenvolvimento
Placa branca	Placa para montagem de circuitos
DHT20	Sensor de temperatura e humidade
TC	Sensor de temperatura
Cabo USB	Cabo USB-A para USB-C (1m)
Display	TFT RGB 0.96" com leitor de cartões SD integrado
SD Card	Cartão SD de 16GB
Pot. 10 KOhm	Potenciómetro
R 1 KOhm	Resistência
LED Vermelho	Díodo emissor de luz
<i>Switch</i>	Botão de pressão
<i>knob</i>	Pino para o potenciómetro

INFO: As resistências e fios de ligação são de acesso livre, não carecendo requisição.

3.1.1 ESP32-C6-DevKitC-1

O ESP32-C6-DevKitC-1 é baseada no módulo ESP32-C6-WROOM-1(U), um módulo de uso geral com 8 MB de flash SPI. A maior parte dos pinos de I/O são disponibilizados em *headers* na parte inferior da placa, facilitando a conexão de periféricos. O desenvolvimento com esta placa pode ser realizado conectando periféricos através de fios *jumper* ou montando o ESP32-C6-DevKitC-1 numa placa branca e ligando aí os periféricos. A vista isométrica da placa está representada na Figura 1.

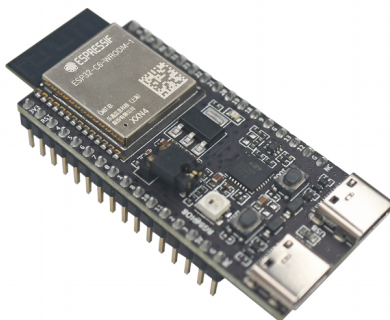


Figura 1: ESP32-C6-DevKitC-1: Vista Isométrica

O ESP32-C6-DevKitC-1 integra funções completas de Wi-Fi 6 na banda de 2,4 GHz, Bluetooth 5 e IEEE 802.15.4 (Zigbee 3.0 e Thread 1.3).

Todos os pinos *General Purpose Input/Output* (GPIO) disponíveis (exceto o barramento SPI para a *flash*) são disponibilizados nos *headers* da placa. A placa também possui um regulador de tensão *low-dropout* (LDO) de 5 V para 3,3 V, um LED de *power ON* e um conversor USB<->UART que suporta taxas de transferência até 3 Mbps.

O ESP32-C6-DevKitC-1 possui uma porta USB Type-C compatível com USB 2.0, capaz de transferências até 12 Mbps. Essa porta é usada para alimentação da placa, gravação de *firmware* e comunicação com o módulo usando protocolos USB, bem como para depuração *Joint Test Action Group* (JTAG).

A placa também possui um botão de *download* (*Boot*) e um botão de reinicialização (*Reset*), além de um LED RGB endereçável acionado pelo GPIO8. Há ainda um conector J5 usado para medição de corrente. A Figura 2 representa e identifica estes elementos na placa.

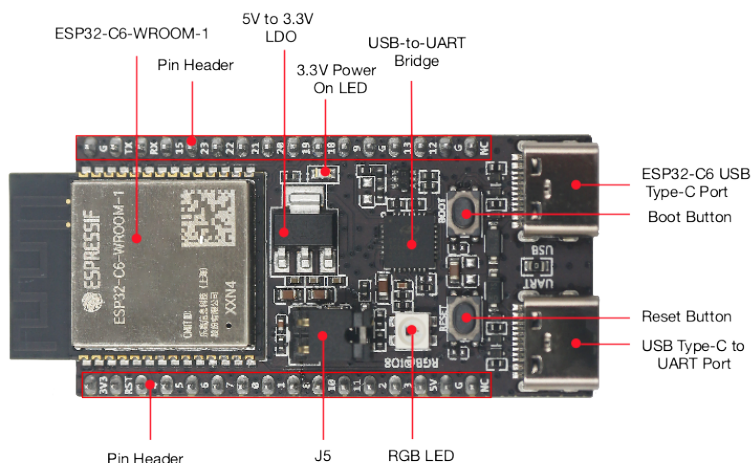


Figura 2: Elementos fundamentais do ESP32-C6-DevKitC-1

Os *headers* do ESP32-C6-DevKitC-1 permitem acesso a sinais digitais, interfaces de comunicação série tais como UART, I2C e SPI, além de pinos analógicos para leitura de sensores. São também disponibilizados pinos dedicados a funções especiais, tais como *Pulse Width Modulation* (PWM) para controle de motores e servos, além de entradas e saídas de propósito geral (GPIO) que podem ser programadas conforme as necessidades.

ESP32-C6-DevKitC-1

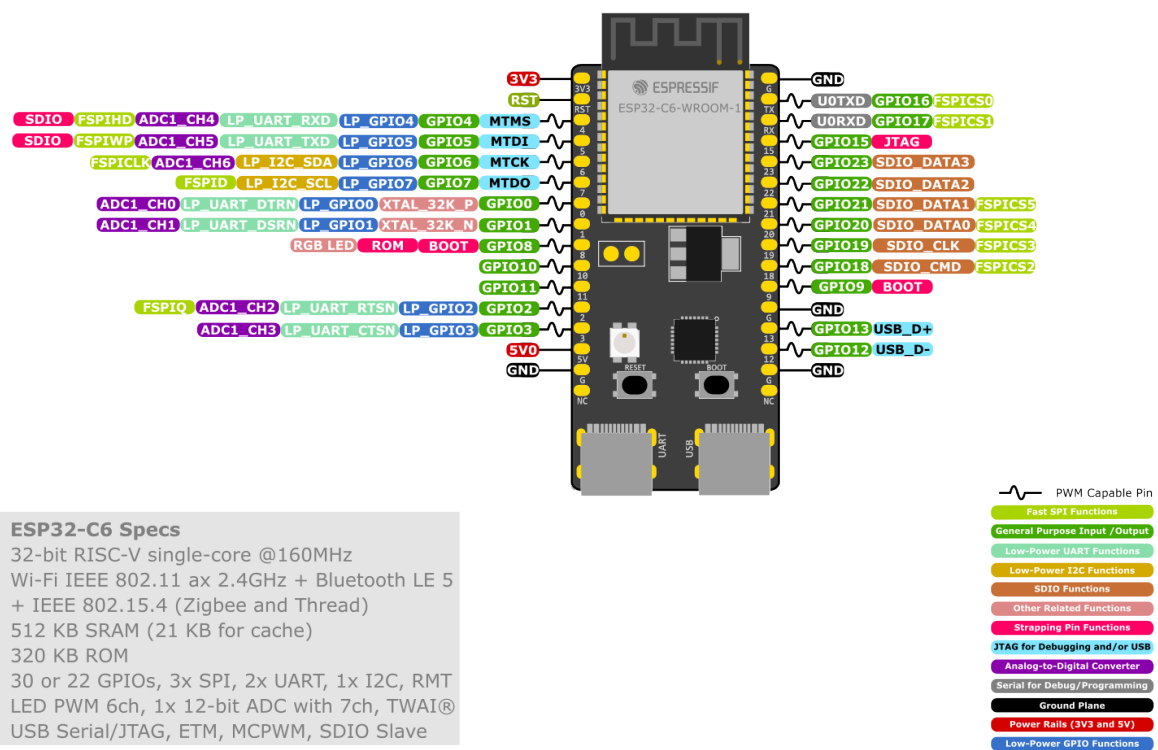


Figura 3: Especificação dos pinos do ESP32-C6-DevKitC-1

J1

No.	Name	Type ¹	Function
1	3V3	P	3.3 V power supply
2	RST	I	High: enables the chip; Low: disables the chip.
3	4	I/O/T	MTMS ³ , GPIO4, LP_GPIO4, LP_UART_RXD, ADC1_CH4, FSPiHD
4	5	I/O/T	MTDI ³ , GPIO5, LP_GPIO5, LP_UART_TXD, ADC1_CH5, FSPiWP
5	6	I/O/T	MTCK, GPIO6, LP_GPIO6, LP_I2C_SDA, ADC1_CH6, FSPiCLK
6	7	I/O/T	MTDO, GPIO7, LP_GPIO7, LP_I2C_SCL, FSPiD
7	0	I/O/T	GPIO0, XTAL_32K_P, LP_GPIO0, LP_UART_DTRN, ADC1_CH0
8	1	I/O/T	GPIO1, XTAL_32K_N, LP_GPIO1, LP_UART_DSRN, ADC1_CH1
9	8	I/O/T	GPIO8 ^{2 3}
10	10	I/O/T	GPIO10
11	11	I/O/T	GPIO11
12	2	I/O/T	GPIO2, LP_GPIO2, LP_UART_RTSN, ADC1_CH2, FSPiQ
13	3	I/O/T	GPIO3, LP_GPIO3, LP_UART_CTSN, ADC1_CH3
14	5V	P	5 V power supply
15	G	G	Ground
16	NC	-	No connection

J3

No.	Name	Type	Function
1	G	G	Ground
2	TX	I/O/T	U0TXD, GPIO16, FSPiC50
3	RX	I/O/T	U0RXD, GPIO17, FSPiC51
4	15	I/O/T	GPIO15 ³
5	23	I/O/T	GPIO23, SDIO_DATA3
6	22	I/O/T	GPIO22, SDIO_DATA2
7	21	I/O/T	GPIO21, SDIO_DATA1, FSPiC55
8	20	I/O/T	GPIO20, SDIO_DATA0, FSPiC54
9	19	I/O/T	GPIO19, SDIO_CLK, FSPiC53
10	18	I/O/T	GPIO18, SDIO_CMD, FSPiC52
11	9	I/O/T	GPIO9 ³
12	G	G	Ground
13	13	I/O/T	GPIO13, USB_D+
14	12	I/O/T	GPIO12, USB_D-
15	G	G	Ground
16	NC	-	No connection

3.2 Plataforma ESP-IDF

A plataforma ESP-IDF (*Espressif IoT Development Framework*) é uma opção popular para o desenvolvimento de soluções IoT baseadas no módulo ESP32-C6-DevKitC-1, fornecendo um ambiente de desenvolvimento robusto, abrangente, e alinhado com as capacidades do módulo.

O ESP-IDF é projetado especificamente para tirar proveito dos recursos do ESP32-C6, oferecendo um conjunto abrangente de *drivers*, bibliotecas e exemplos. A sua integração permite o foco do desenvolvimento no *core* das aplicações IoT, evitando as complexidades de baixo nível relacionadas com a configuração e controlo dos periféricos.

Adicionalmente, esta plataforma é *open-source* e bem documentada, contando com uma comunidade ativa de utilizadores e *developers*, o que facilita a descoberta de soluções, a resolução de problemas e a adoção de práticas comprovadas durante o ciclo de desenvolvimento.

Outra vantagem significativa do ESP-IDF é o seu suporte nativo a recursos avançados de conectividade, tais como Wi-Fi 6, Bluetooth 5 e protocolos de rede de baixa potência, como Zigbee e Thread. Essa abordagem integrada simplifica enormemente a implementação de aplicações IoT que exigem acesso a múltiplas tecnologias de comunicação.

3.2.1 Instalação e configuração da plataforma ESP-IDF

O primeiro passo para iniciar a jornada com o módulo ESP32-C6-DevKitC-1 é a instalação do ESP-IDF que irá permitir editar o *firmware* da aplicação IoT, compilar o código fonte, programá-lo no módulo e realizar a respetiva depuração e teste.

Para o efeito, deve seguir rigorosamente as indicações disponibilizadas na página [Get Started para o módulo ESP32-C6 da Espressif](#) onde encontra um guia passo-a-passo de como instalar a plataforma.

NOTA 1: Assegure-se que instala os pré-requisitos devidamente.

A forma recomendada de instalação nesta disciplina é através da [Extensão para o VSCode](#) e sugere-se o uso da versão ESP-IDF 5.5.1 ou superior.

No Visual Studio Code previamente instalado deverá clicar no ícone de Extensões da Barra de Actividade (ou através do comando View -> Show Extensions - Ctrl+Shift+X), procurar a extensão

“ESP-IDF” e prosseguir com a respetiva instalação. As Figuras 4, 5 e 6 resumem o processo a adoptar.

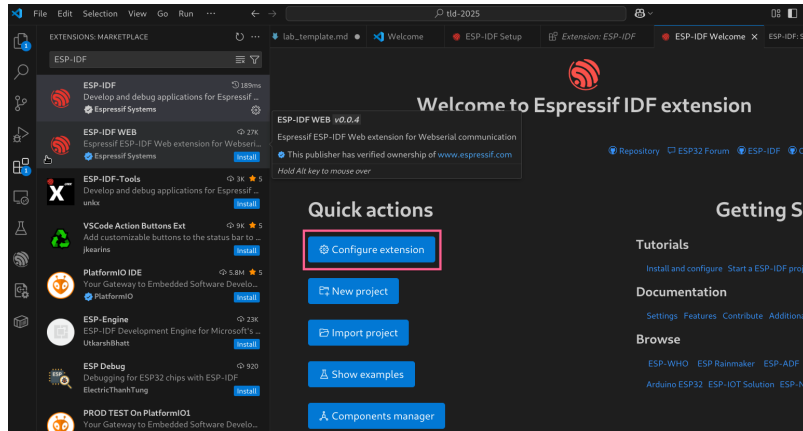


Figura 4: Configuração do ESP-IDF

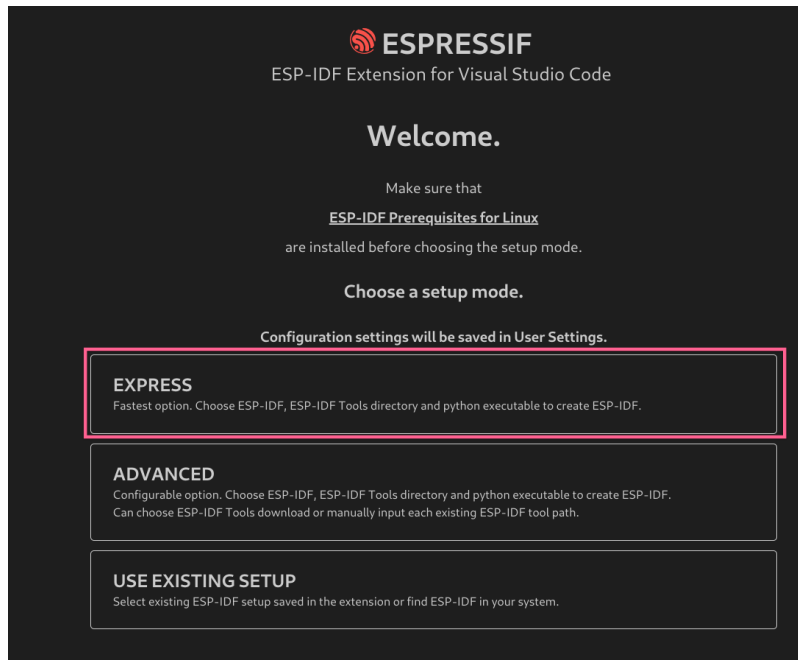


Figura 5: Configuração *Express*

Por fim, a secção do “Using the ESP-IDF Extension for VS Code” disponibiliza um pequeno tutorial sobre como usar a extensão, concretamente como interpretar os ícones da barra do fundo da janela do VS Code que são disponibilizados.

3.3 Aplicação “Hello World”

Depois de ter concluído a instalação do ambiente de desenvolvimento o próximo passo é a criação de uma aplicação básica para testar todo o ecossistema.

Para o efeito, no menu de criação novos projetos, crie um projeto com base no exemplo “Hello

ESPRESSIF
ESP-IDF Extension for Visual Studio Code

Git version: 2.30.2

Select download server:
Github

Show all ESP-IDF tags

Select ESP-IDF version:
v5.5.1 (release version)

Enter ESP-IDF container directory:
/home/parallels/esp

Enter ESP-IDF Tools directory (IDF_TOOLS_PATH):
/home/parallels/esp/esp-idf/tools

Select Python version:
/usr/bin/python3

Install

Figura 6: Parâmetros de configuração

World”. As Figuras 7 e 8 ilustram o processo.

NOTA 2: Caso não selecione devidamente o *target* ESP32-C6 os binários gerados não serão compatíveis com a placa.

Após ter criado o projeto, compile-o, descarregue-o para a placa e teste-o, tentando interpretar o código e o respetivo funcionamento.

4 Parte 2: Input/Output Básico

Para esta parte do trabalho considere a [API de Referência para os periféricos do ESP32-C6](#) juntamente com o [ESP32-C6 Technical Reference Manual](#).

Implemente na placa branca recebida os três circuitos especificados na Figura 9.

ATENÇÃO! Seja prudente e confirme as ligações na placa branca antes de alimentar o ESP32. Seja particularmente cauteloso a verificar as ligações ao kit ESP32-C6.

4.1 Controlo de um LED

O passo mais elementar de qualquer sistema *embedded* é o de programar uma saída digital para alternar entre o estado “1” e o estado “0” com um determinado período.

Em seguida é facultado o “esqueleto” de um programa funcional que, conjugado com o circuito da Figura 9a) permite ativar periodicamente o respetivo LED.

New Project

Project Name
hello

Enter Project directory
/home/parallels /hello

Choose ESP-IDF Target (IDF_TARGET)
esp32c6

Choose ESP-IDF Board
ESP32-C6 chip (via builtin USB-JTAG)

Choose serial port
/dev/ttyS0

Add your ESP-IDF Component directory
/home/parallels/esp/v5.5.1/esp-idf/components

[Choose Template](#)

Figura 7: Configuração base da aplicação

New Project

ESP-IDF [Create project using template hello_world](#)

Search Template By Name

Supported Targets	ESP32	ESP32-C2	ESP32-C3	ESP32-C5	ESP32-C6	ESP32-C61	ESP32-H2	ESP32-H21
get-started								
blink								
hello_world								
bluetooth								
bluedroid								
Bluedroid_Beacon								
Bluedroid_Connection								
Bluedroid_GATT_Server								
nimble								
NimBLE_Beacon								
NimBLE_Connection								
NimBLE_GATT_Server								
NimBLE_Security								
ble								
ble_throughput								
throughput_client								
throughput_server								
ble_ancs								

Hello World Example

Starts a FreeRTOS task to print "Hello World".

(See the README.md file in the upper level 'examples' directory for more information about examples.)

How to use example

Follow detailed instructions provided specifically for this example.

Select the instructions depending on Espressif chip installed on your development board:

- [ESP32 Getting Started Guide](#)
- [ESP32-S2 Getting Started Guide](#)

Example folder contents

The project **hello_world** contains one source file in C language **hello_world_main.c**. The file is located in folder **main**.

ESP-IDF projects are built using CMake. The project build configuration is contained in **CMakeLists.txt** files that provide set of directives and instructions describing the project's source files and targets (executable, library, or both).

Figura 8: Criação da aplicação a partir do template “Hello World”

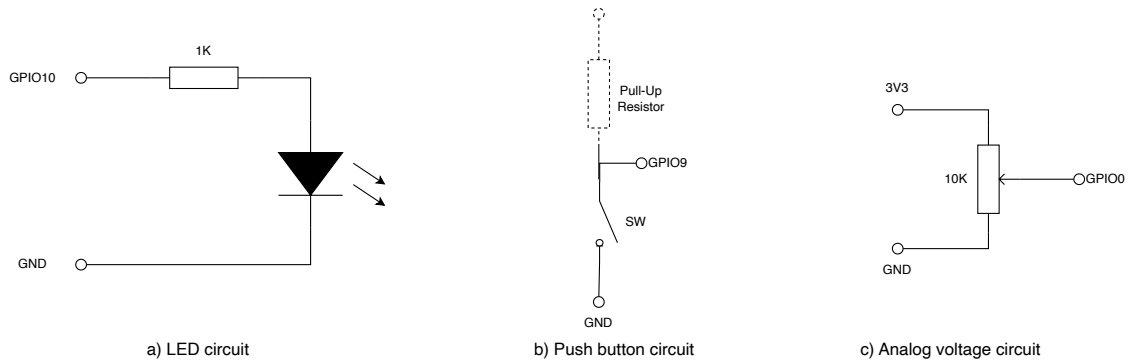


Figura 9: Circuitos de interface com o kit ESP32-C6

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"

#define BLINK_GPIO  GPIO_NUM_10    // GPIO10 on ESP32-C6

void app_main(void)
{
    // Configure GPIO10 as output
    gpio_config_t io_conf = {
        .pin_bit_mask = 1ULL << BLINK_GPIO, // bit mask for GPIO10
        .mode = GPIO_MODE_OUTPUT,
        .pull_up_en = GPIO_PULLUP_DISABLE,
        .pull_down_en = GPIO_PULLEDOWN_DISABLE,
        .intr_type = GPIO_INTR_DISABLE
    };
    gpio_config(&io_conf);

    bool level = false;

    while (1) {
        gpio_set_level(BLINK_GPIO, level);
        level = !level; // invert for next toggle
        vTaskDelay(pdMS_TO_TICKS(1000)); // delay 1 second
    }
}
```

TROUBLESHOOTING: Caso encontre problemas em colocar o programa a funcionar devido à não satisfação de dependências pode editar o ficheiro `CMakeLists.txt` que está na sua pasta raiz (a que contém a sua “main”) usando o bloco de código seguinte. Como não há `REQUIRES` nem `PRIV_REQUIRES`, não se declara dependências explícitas de nenhum outro componente, logo todos os componentes definidos no projeto serão compilados e ligados normalmente. Em desenvolvimento é recomendado que as dependências sejam identificadas de forma explícita recorrendo a `REQUIRES` e `PRIV_REQUIRES` no `idf_component_register`.

```
idf_component_register(SRCS "main.c"
                      INCLUDE_DIRS ".")
```

Consultando a secção [GPIO & RTC GPIO](#) da [API de Referência](#), analise o programa e teste-o na placa observando o LED a “piscar” com um período de 2 segundos.

QUESTÃO 1: Altere o código para um período de refrescamento de 20 milissegundos e observe no osciloscópio o sinal produzido pela placa, confirmando quer a frequência pretendida quer o *duty-cycle* do sinal.

QUESTÃO 2: Reponha o período original (2 segundos) e acrescente uma impressão periódica do estado do LED de modo a que seja impressa para o terminal a seguinte informação.

```
~$ GPIO 10 set to 1
~$ GPIO 10 set to 0
~$ GPIO 10 set to 1
~$ GPIO 10 set to 0
~$ ...
```

4.2 Leitura do estado de um botão de pressão (*polling*)

Tendo como ponto de partida o circuito da Figura 9b) pretende-se agora ler o estado de um botão de pressão.

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"

#define INPUT_GPIO    GPIO_NUM_9    // choose an available GPIO on your board

void app_main(void)
{
    // Configure the GPIO as input with pull-up enabled
    gpio_config_t io_conf = {
        .pin_bit_mask = 1ULL << INPUT_GPIO, // bit mask for selected GPIO
        .mode = GPIO_MODE_INPUT,
        .pull_up_en = GPIO_PULLUP_ENABLE,    // enable internal pull-up
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .intr_type = GPIO_INTR_DISABLE
    };
    gpio_config(&io_conf);

    while (1) {
        int level = gpio_get_level(INPUT_GPIO); // read the pin state (0 or 1)

        // INCLUA AQUI O CODIGO PARA IMPRIMIR O ESTADO DO BOTAO

        vTaskDelay(pdMS_TO_TICKS(1000)); // check every 1s
    }
}
```

QUESTÃO 3: Analise as opções de configuração e complete o código de modo a que se visualize o estado do botão de pressão no monitor de acordo com o seguinte exemplo:

```
~$ GPIO 9 read 1
~$ GPIO 9 read 1
~$ GPIO 9 read 1
~$ GPIO 9 read 0
~$ ...
```

QUESTÃO 4: Prossiga com a compilação e execução do código desenvolvido e verifique o funcionamento. O que pode dizer sobre a lógica de funcionamento do botão de pressão? É direta ou invertida?

QUESTÃO 5: Qual o objetivo da resistência ligada ao botão de pressão e identificada no circuito da Figura 9b) a tracejado? Porque é que embora não tenha montado essa resistência, internamente ela é ligada?

4.3 Leitura do estado de um botão de pressão (*interrupt*)

Quando se utilizam interrupções em sistemas *embedded*, surge sempre o desafio de transferir informação de forma segura entre a rotina de serviço da interrupção (ISR) e a aplicação principal. Implementar operações como imprimir dentro de uma ISR não é recomendado, pois pode atrasar outros eventos importantes. As filas (queues) oferecem um mecanismo eficiente e seguro para esta comunicação. A ISR pode rapidamente colocar na fila um pequeno dado, como um sinal de evento ou uma leitura de sensor, e regressar de imediato. Mais tarde, uma tarefa do FreeRTOS, a correr fora do contexto da interrupção, retira essa informação da fila e processa-a sem restrições de tempo. Desta forma, as ISRs permanecem leves, ao mesmo tempo que se garante uma transferência de dados fiável e sincronizada entre os eventos de interrupção e a lógica da aplicação.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"
#include "driver/gpio.h"

#define INPUT_GPIO    GPIO_NUM_9    // example input pin

static QueueHandle_t gpio_evt_queue = NULL;

// ISR handler
static void IRAM_ATTR gpio_isr_handler(void *arg)
{
    uint32_t gpio_num = (uint32_t) arg;
    // Send GPIO number to the queue (from ISR context)
    xQueueSendFromISR(gpio_evt_queue, &gpio_num, NULL);
}

// Task that handles GPIO events
static void gpio_task(void *arg)
{
    uint32_t io_num;
    for(;;) {
        if (xQueueReceive(gpio_evt_queue, &io_num, portMAX_DELAY)) {
            // INCLUA AQUI O CODIGO PARA IMPRIMIR O NUMERO DO GPIO ORIGEM DA INTERRUPCAO
        }
    }
}

void app_main(void)
{
    // Configure the GPIO as input with pull-up, interrupt on falling edge
    gpio_config_t io_conf = {
        .pin_bit_mask = 1ULL << INPUT_GPIO,
        .mode = GPIO_MODE_INPUT,
        .pull_up_en = GPIO_PULLUP_ENABLE,
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .intr_type = GPIO_INTR_NEGEDGE
    };
    gpio_config(&io_conf);

    // Create a queue to handle GPIO events
    gpio_evt_queue = xQueueCreate(10, sizeof(uint32_t));
```

```

// Start the task that will process GPIO events
xTaskCreate(gpio_task, "gpio_task", 2048, NULL, 10, NULL);

// Install GPIO ISR service
gpio_install_isr_service(0);

// Hook ISR handler for specific GPIO
gpio_isr_handler_add(INPUT_GPIO, gpio_isr_handler, (void*) INPUT_GPIO);

printf("Waiting for GPIO %d falling edge interrupts...", INPUT_GPIO);
}

```

QUESTÃO 6: Consulte e analise as APIs de [GPIO & RTC GPIO](#) e da [Queue](#). Estude o código fornecido e complete o programa de modo a imprimir a seguinte informação sempre que ocorra um flanco descendente no sinal do botão de pressão.

```

~$ Interrupt on GPIO 9
~$ Interrupt on GPIO 9
~$ Interrupt on GPIO 9
~$ Interrupt on GPIO 9
~$ ...

```

4.4 Controlo de uma saída de PWM

A biblioteca LEDC do ESP-IDF é uma ferramenta de eleição para gerar sinais PWM de forma precisa e eficiente em microcontroladores da família ESP32. Através dela é possível controlar a intensidade de um LED ajustando o ciclo ativo do sinal (*duty-cycle*), permitindo, por exemplo, criar efeitos de *fade* suave ou simplesmente regular o brilho em diferentes níveis. No caso de um LED, o LEDC torna possível transformar um simples pino digital num regulador de intensidade, abrindo caminho para interfaces mais dinâmicas e interativas em projetos de IoT e sistemas embebidos.

```

#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/ledc.h"

#define LED_GPIO      GPIO_NUM_10
#define LEDC_CHANNEL  LEDC_CHANNEL_0
#define LEDC_TIMER     LEDC_TIMER_0
#define LEDC_MODE      LEDC_LOW_SPEED_MODE
#define LEDC_DUTY_RES  LEDC_TIMER_13_BIT // 13-bit resolution
#define LEDC_FREQUENCY 5000              // 5 kHz PWM

void app_main(void)
{
    // 1. Configure timer
    ledc_timer_config_t ledc_timer = {
        .speed_mode     = LEDC_MODE,
        .timer_num      = LEDC_TIMER,
        .duty_resolution = LEDC_DUTY_RES,
        .freq_hz         = LEDC_FREQUENCY,
        .clk_cfg         = LEDC_AUTO_CLK
    };
    ledc_timer_config(&ledc_timer);

    // 2. Configure channel
    ledc_channel_config_t ledc_channel = {
        .gpio_num = LED_GPIO,
        .speed_mode = LEDC_MODE,
        .channel = LEDC_CHANNEL,
        .intr_type = LEDC_INTR_DISABLE,
    };
}

```

```

        .timer_sel = LEDC_TIMER,
        .duty      = 0,
        .hpoint    = 0
    };
    ledc_channel_config(&ledc_channel);

    uint32_t max_duty = (1 << LEDC_DUTY_RES) - 1;
    uint32_t duty_10 = max_duty / 10;

    while (1) {
        ledc_set_duty(LEDC_MODE, LEDC_CHANNEL, duty_10);
        ledc_update_duty(LEDC_MODE, LEDC_CHANNEL);
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

```

QUESTÃO 7: Estude a [API LEDC](#) e analise o funcionamento do código do exemplo.

QUESTÃO 8: Altere o código para diferentes valores de *duty-cycle* e observe o sinal de *drive* do LED no osciloscópio, confirmando-o. Como justifica a frequência usada para o PWM?

QUESTÃO 9: Altere o código de modo a que seja possível observar o LED com um efeito de “respirar”, ou seja, vai incrementalmente aumentando o brilho até alcançar o máximo e depois diminui até chegar ao mínimo, repetindo este processo indefinidamente.

4.5 Configuração e utilização da ADC no modo One-shot

O modo de funcionamento oneshot da ADC aplica-se quando é necessário obter leituras pontuais e precisas de sinais analógicos, sem manter a conversão ativa de forma contínua. Este modo permite ao processador solicitar uma medição apenas quando necessário, reduzindo o consumo de energia e libertando recursos do sistema, o que é especialmente importante em dispositivos IoT alimentados a bateria. Além disso, o oneshot simplifica a integração com tarefas específicas, como a monitorização de sensores em intervalos definidos ou a leitura de valores críticos em resposta a eventos, garantindo eficiência e precisão no tratamento de sinais analógicos dentro de arquiteturas *embedded*.

```

#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_adc/adc_oneshot.h"

#define ADC_CHANNEL    ADC_CHANNEL_0    // GPIO0 (check pin mapping for ESP32-C6)
#define ADC_UNIT       ADC_UNIT_1      // Use ADC1
#define ADC_ATTEN      ADC_ATTEN_DB_12 // 12 dB, ~1.1 V full-scale

void app_main(void)
{
    // ADC Oneshot driver handle
    adc_oneshot_unit_handle_t adc1_handle;
    adc_oneshot_unit_init_cfg_t init_config1 = {
        .unit_id = ADC_UNIT,
    };
    adc_oneshot_new_unit(&init_config1, &adc1_handle);

    // Configure channel
    adc_oneshot_chan_cfg_t config = {
        .atten = ADC_ATTEN,
        .bitwidth = ADC_BITWIDTH_DEFAULT, // default = 12-bit
    };
    adc_oneshot_config_channel(adc1_handle, ADC_CHANNEL, &config);
}

```

```

while (1) {
    int adc_raw = 0;
    adc_oneshot_read(adc1_handle, ADC_CHANNEL, &adc_raw);

    // Convert raw value to voltage (approximate, no calibration)
    float voltage = (adc_raw / 4095.0f) * (1.1f/0.25f);

    // INCLUA AQUI O CODIGO PARA IMPRIMIR O VALOR DIGITAL LIDO E O CORRESPONDENTE ANALOGICO

    vTaskDelay(pdMS_TO_TICKS(1000));
}
}

```

QUESTÃO 10: Estude o [driver Oneshot da ADC](#) e complete o código de modo a que seja impressa a seguinte informação a cada segundo:

```

~$ ADC Raw: 4095, Voltage: 3.899 V
~$ ADC Raw: 4095, Voltage: 3.899 V
~$ ADC Raw: 4095, Voltage: 3.899 V
~$ ADC Raw: 4095, Voltage: 3.899 V
~$ ...

```

QUESTÃO 11: Observe o valor de tensão analógico e compare-o com o valor obtido pelo seu programa. Como justifica a discrepância?

QUESTÃO 12: Implemente um mecanismo básico de calibração e teste-o na prática, avaliando o seu funcionamento.

QUESTÃO 13: Estude a API da ADC e identifique uma alternativa ao método empregue e implemente-a, comparando com o resultado da calibração básica na questão anterior.

QUESTÃO 14: O ESP-IDF permite criar logs para as diferentes funcionalidades através da [Logging library](#). Repita os exercícios anteriores substituindo o uso da função `printf` pela macro `ESP_LOGI`.

5 Acknowledgements

Originally authored by [Paulo C. Bartolomeu](#)